

Part 01: Questions

1. What is the shortcut to comment and uncomment a selected block of code in Visual Studio?

* Comment: Ctrl + K, Ctrl + C

* Uncomment: Ctrl + K, Ctrl + U

2 Explain the difference between a runtime error and a logical error with examples.

* Runtime Error: خطأ يحدث أثناء تشغيل البرنامج ويؤدي إلى توقفه فوراً (Crash).

* Example: القسمة على صفر $int\ result = 10 / 0$ ؛ أو إدخال نص بدل رقم أثناء الـ Parsing.

* Logical Error: البرنامج يعمل وينفذ بدون أي أخطاء أو توقف، لكن الناتج النهائي خاطئ بسبب منطق الكود غير الصحيح.

* Example: حساب مساحة المستطيل بالطرح بدلاً من الضرب $int\ area = width - height$.

3. Why is it important to follow naming conventions such as PascalCase in C#?

* تزيد من قراءة الكود وتنظيمه (Readability & Maintainability).

* تضمن الالتزام بالـ Clean Code ومعايير شركة Microsoft لتسهيل العمل الجماعي داخل الفريق.

4. Explain the difference between value types and reference types in terms of memory allocation.

* Value Types (e.g., int, double, struct)

* تخزن القيم الفعلية مباشرة في ذاكرة الـ Stack.

* Reference Types (e.g., class, string, array)

* تخزن العنوان (Address/Reference) في ذاكرة الـ Stack، بينما تُخزن البيانات الفعلية (Object) في ذاكرة الـ Heap.

5. What will be the output of the following code? Explain why:

```
int a = 2, b = 7;
```

```
Console.WriteLine(a % b);
```

* Output: 2

* Explanation: المعامل % يحسب باقي القسمة (Remainder). بما أن الرقم الأول (٢) أصغر من الرقم الثاني (٧)، فإن ناتج القسمة الصحيحة هو ٠ وباقي القسمة هو ٢.

6. How does the && (logical AND) operator differ from the & (bitwise AND) operator?

* && (Short-Circuit Logical AND): يفحص الشرط الثاني فقط إذا كان الشرط الأول صحيحاً (True). أما إذا كان الشرط الأول خطأ (False) يتوقف فوراً.

* & (Bitwise / Boolean AND): يفحص كلا الشرطين دائماً بغض النظر عن نتيجة الشرط الأول، أو يُستخدم للمقارنة على مستوى الـ Bits للأنظمة الرقمية.

7. Why is explicit casting required when converting a double to an int?

* لأن التحويل من double إلى int يعتبر Narrowing Conversion يترتب عليه فقدان بيانات (Data Loss) لشرائح الأرقام العشرية. تفرض لغة #C كتابة التحويل الصريح (int) لتأكيد موافقة المبرمج على مسح الكسر.

8. What exception might occur if the input is invalid and how can you handle it?

* Exception: يحدث FormatException (إذا كان النص المدخل يحتوي حروفاً) أو OverflowException (إذا كان الرقم أكبر من نطاق من نطاق int).

* How to Handle: استخدام دالة int.TryParse () بدلاً من int.Parse (), أو إحاطة الكود بـ try-catch block.

9. Given the code below, what is the value of x after execution? Explain why:

```
int x = 5;
```

```
int y = ++x + x++;
```

* Value of x: 7 (القيمة النهائية لـ y ستكون ١٢)

* Explanation:

* ++x (Prefix): تزيد قيمة x أولاً من ٥ إلى ٦ وتُستخدم في العملية.

* ++x (Postfix): تُستخدم قيمة x الحالية وهي ٦ في العملية الحسابية، ثم تُزاد قيمة x بعد ذلك لتصبح ٧.

Part 02

٢. Difference between Compiled and Interpreted Languages & What about C#?

* ++Compiled Languages (e.g., C, Machine Code): يتم ترجمة الكود المصدر دفعة واحدة إلى لغة الآلة (Machine Code) المباشرة قبل التشغيل، مما يجعلها سريعة جداً.

* Interpreted Languages (e.g., Python, JS, Interpreter): يتم تنفيذ الكود سطرًا بسطرًا أثناء التشغيل عبر مفسر (Interpreter)، وهي أبطأ نسبياً.

* C# (Hybrid / Two-Step Compilation):

* يُترجم كود #C بواسطة محرر Roslyn إلى لغة وسيطة تسمى IL (Intermediate Language).

* أثناء التشغيل، يقوم JIT Compiler (Just-In-Time) داخل بيئة CLR بتحويل الـ IL إلى Machine Code يناسب المعالج.

٣. Comparison: Implicit, Explicit, Convert, and Parse Casting

Casting Type	Description	Handling null / Invalid Data	Example
Implicit	تحويل تلقائي آمن بدون فقد بيانات (Small \rightarrow Big).	آمن كلياً	<code>double d = 10</code> ؛
Explicit	تحويل يدوي يُلزم المبرمج بالتأكد (type) مع احتمال فقد بيانات.	يقطع الكسر العشري	<code>int x = 10.5</code> ؛
Convert	دالة تحويل بين الأنواع المختلفة تحول قيم الـ null بأمان.	<code>Convert.ToDouble(null)</code> يرجع ٠	<code>Convert.ToInt32("10")</code> ؛
Parse	تحويل النص (string) حصراً إلى نوع بيانات رقمي.	يضرب Exception لو النص null أو غير رقمي	<code>int.Parse("10")</code> ؛

Part 03: Bonus

4. Self-Study Report Template

* Topic: C# Type System, Memory Management, and Data Conversions.

* Key Learnings:

- * Difference between Value Types (Stack) and Reference Types (Heap).
- * Safe data conversion techniques using TryParse to avoid runtime crashes.
- * Understanding the .NET execution model (CLR & JIT Compilation.)

5. What is meant by C# is Managed Code?

معناه أن كود C# لا يتعامل مع الذاكرة والمعالج بشكل مباشر، بل يتم إدارته تنفيذه بالكامل بواسطة بيئة التشغيل CLR (Common Language Runtime) التي توفر:

* إدارة الذاكرة التلقائية (Garbage Collection).

* فحص الأمان وفحص أنواع البيانات (Type Safety & Security).

* التعامل مع الأخطاء والمستثنيات (Exception Handling).

6. What is meant by "struct is considered like class before?"

في المفاهيم البرمجية القديمة (أو في لغات مثل ++C)، كان الـ struct والـ class متطابقين تقريباً كـ Data Containers.

أما في C# الحديثة:

* struct: أصبح يمثل Value Type يُحجز في الـ Stack وخفيف الحجم.

* class: يمثل Reference Type كامل الميزات (Inheritance, Polymorphism) يُحجز في الـ Heap

1. Scientific Notation / FPU

- Scientific Notation (الصيغة العلمية):

- هي طريقة لتمثيل الأرقام الكبيرة جداً أو الصغيرة جداً باستخدام الأسس (مثل $10^3 = 1500$ و $10^{-3} = 0.001$ والتي تساوي 10^{-3}).

- تُستخدم مع أنواع البيانات ذات النقطة العائمة (Floating-Point) مثل `float` و `double`.

- FPU (Floating-Point Unit):

- وحدة المعالجة بالرياضيات العائمة، وهي جزء مادي (Hardware Component) داخل المعالج (CPU) متخصص في تنفيذ العمليات الحسابية للأرقام العشرية والكسور بسرعة عالية جداً.

2. Bitwise Operators << self

- Bitwise Operators (معاملات البتات):

هي المعاملات التي تتعامل مع الأرقام على مستوى الـ Bits (0 و 1) زي & (AND)، | (OR)، ^ (XOR)، و ~ (NOT).

3. checked / unchecked << self

في لغة C#، تُستخدم هذه الكلمات المفتاحية للتحكم في ظاهرة الـ **Overflow** (تجاوز السعة الرقمية):

- `:checked`

- يخبر الكومبايلر بفحص العمليات الحسابية؛ وإذا تجاوز الناتج الحد الأقصى لنوع البيانات (مثل تجاوز `int` في `2,147,483,647`) يرمي خطأ فوراً من نوع `OverflowException`.

- unchecked:

- يتجاهل تجاوز السعة، وإذا زاد الرقم عن الحد الأقصى يلف ويرجع لأقل قيمة بالسالب من غير ما يرمي أي Exception (وهذا هو الوضع الافتراضي في C# لزيادة السرعة).

4.Parse ? / null ? self

- Parse:

- تحويل النص (string) إلى نوع رقمي (مثل "123" Parse ("int").

- **تنبيه:** إذا كانت القيمة المدخلة null أو تحتوي على حروف، تسبب خطأ واستثناء (FormatException أو ArgumentNullException).

- null Handling:

- لمعالجة قيم null بأمان، يُفضل استخدام Convert.ToInt32 () (التي ترجع ٠ عند الـ null) أو int.TryParse () لتفادي توقف البرنامج.